METRO ORANGE LINE

LAHORE METRO RAIL TRANSIT

AUTOMATED THIRD RAIL INSPECTION SYSTEM USING IMAGE PROCESSING

Phase 1 — Final Technical Deliverable Report

Version 1.0 | April 2026

Prepared for	**METRO Train Station Management**
Prepared by	Umer Shahid — Lecturer, EE, UET Lahore
Date	June 2025
Classification	**Confidential — Internal Use Only**

Table of Content

Executive Summary **3**
1. Project Background & Objectives **4**
1.1 Context 4
1.2 Phase 1 Scope 4
1.3 Deliverables Mapping 4
2. Technical Architecture **5**
2.1 System Overview 5
2.2 The 2-Script Architecture 5
2.3 Detection Pipeline — 32-Step Process 5
2.4 Sample Processing Stages 6
3. Dataset & Annotation **8**
3.1 Dataset Composition 8
3.2 Defect Classification Schema 8
4. Visual Results & Analysis Reports **9**
4.1 Side-by-Side Comparisons 9
4.2 Sample Automated Defect Report 10
5. System Performance **10**
6. Source Code & Repository **11**
6.1 Repository Structure 11
6.2 Installation & Dependencies 11
7. Conclusion **12**
Contact & Payment Details **12**

Executive Summary

This report presents the complete Phase 1 deliverables for the Automated Third Rail Inspection System (TRISS) developed for the METRO Orange Line, Lahore. The engagement was scoped as a software-only proof-of-concept phase, leveraging pre-recorded imagery from the Orange Line's third rail infrastructure.

The system was developed by the Maktab e Digital Systems (MEDS) at the University of Engineering & Technology (UET) Lahore, under the technical direction of Mr. Umer Shahid, Lecturer in the Department of Electrical Engineering.

Images Processed	Defect Types Detected	Processing Pipeline Steps	Algorithms Implemented
54	3 Categories	32 Steps	7+ Methods

All six Phase 1 deliverables defined in the original project proposal have been completed and are documented in full within this report. The software system is capable of detecting three classes of third-rail defects — corrosion/dark spots, surface wear patterns, and cracks/linear defects — with severity classification and automated report generation.

The system is immediately ready for demonstration and, upon procurement of the specified hardware components, can be deployed as a real-time on-site inspection tool in Phase 2.

1. Project Background & Objectives

1.1 Context

The third rail of the METRO Orange Line is the sole source of traction power for all rolling stock. Any surface degradation — whether corrosion, mechanical wear, or structural cracking — directly threatens both operational continuity and passenger safety. Historically, inspection has relied on manual walkthroughs during engineering hours, a process that is time-consuming, inconsistent, and dependent on the experience of individual inspectors.

This project was initiated to develop and validate an automated, image-processing-based alternative capable of detecting defects with greater consistency, speed, and documentation quality than manual methods.

1.2 Phase 1 Scope

Phase 1 is strictly a software and proof-of-concept engagement. No physical chassis, embedded deployment, or live track operation is included in this phase. The scope covers:

- Development of a modular Python-based image processing pipeline
- Processing and annotation of a curated dataset of real METRO third-rail imagery
- Generation of visual analysis reports and side-by-side defect comparisons
- A GUI-enabled demo script for real-time or batch playback
- Full technical documentation and packaged source code

1.3 Deliverables Mapping

#	Deliverable	Description	Status
1	**Image Processing Pipeline**	Modular Python-based detection code with multiple algorithms	**Complete**
2	**Dataset Annotation**	54 curated real-world METRO rail images, processed and annotated	**Complete**
3	**Visual Reports**	Side-by-side before/after comparisons + defect summary reports	**Complete**
4	**Demo Scripts**	Video/image playback with real-time or batched analysis GUI	**Complete**
5	**Documentation**	Technical report with implementation details and recommendations	**Complete**
6	**Source Code**	All scripts packaged and published on GitHub	**Complete**

2. Technical Architecture

2.1 System Overview

The Third Rail Defect Detection System (TRDDS) is implemented entirely in **Python 3.8+** using **OpenCV 4.5+** and **NumPy**. The architecture is modular — each processing stage is encapsulated as an independent function, allowing individual algorithms to be substituted or tuned without affecting the rest of the pipeline.

The system operates in two modes: **Batch Processing mode** for analyzing folders of static images, and **Live Webcam mode** for real-time inspection via a connected camera feed. Both modes share the same underlying detection engine.

2.2 The 2-Script Architecture

Script	Mode	Purpose
TRD_training.py	Batch	Processes all images in a folder; outputs annotated images, side-by-side comparisons, and text reports
TRD_video.py	Live / Real-time	GUI-enabled live camera feed with defect overlay, RECORD and CAPTURE buttons; optimized for Raspberry Pi 5

2.3 Detection Pipeline — 32-Step Process

The demo script exposes all 32 stages of the detection pipeline. The key stages are summarized below:

Stage	Step	Description
Pre-processing	**ROI Extraction**	Isolates the rail contact surface using configurable crop percentages, removing irrelevant background
Pre-processing	**Grayscale Conversion**	Converts to single-channel for efficient intensity-based analysis
Pre-processing	**LED Glare Removal**	Detects and inpaints bright LED reflections using adaptive thresholding and morphological masking
Pre-processing	**Bilateral Filtering**	Edge-preserving smoothing to reduce noise while retaining defect boundaries
Corrosion Detection	**Simple + Adaptive Threshold**	Dual thresholding strategy identifies dark oxidized regions across varying lighting conditions
Corrosion Detection	**Morphological Cleanup**	Erosion and dilation operations remove noise and consolidate genuine defect regions
Wear Detection	**High-Pass + Line Detection**	Horizontal/vertical kernel filtering isolates wear-induced surface texture changes
Wear Detection	**Laplacian Edge Analysis**	Second-order derivative detects fine surface irregularities characteristic of mechanical wear
Crack Detection	**Canny Edge Detection**	Multi-threshold Canny identifies sharp discontinuities at two sensitivity levels

Crack Detection	**Hough Line Transform**	Probabilistic Hough transform extracts linear crack geometries from edge maps
Classification	**Contour Analysis**	All detected contours are filtered by area, classified by type, and assigned Low/Medium/High severity
Output	**Final Visualization**	Annotated result image with color-coded bounding boxes and a summary overlay

2.4 Sample Processing Stages

The following images illustrate key stages of the pipeline on a real METRO third-rail image:

Figure 1 — Step 1: Original METRO Third Rail Image (input)

Figure 2 — Step 2: Region of Interest (ROI) Selection — focusing analysis on the rail contact surface

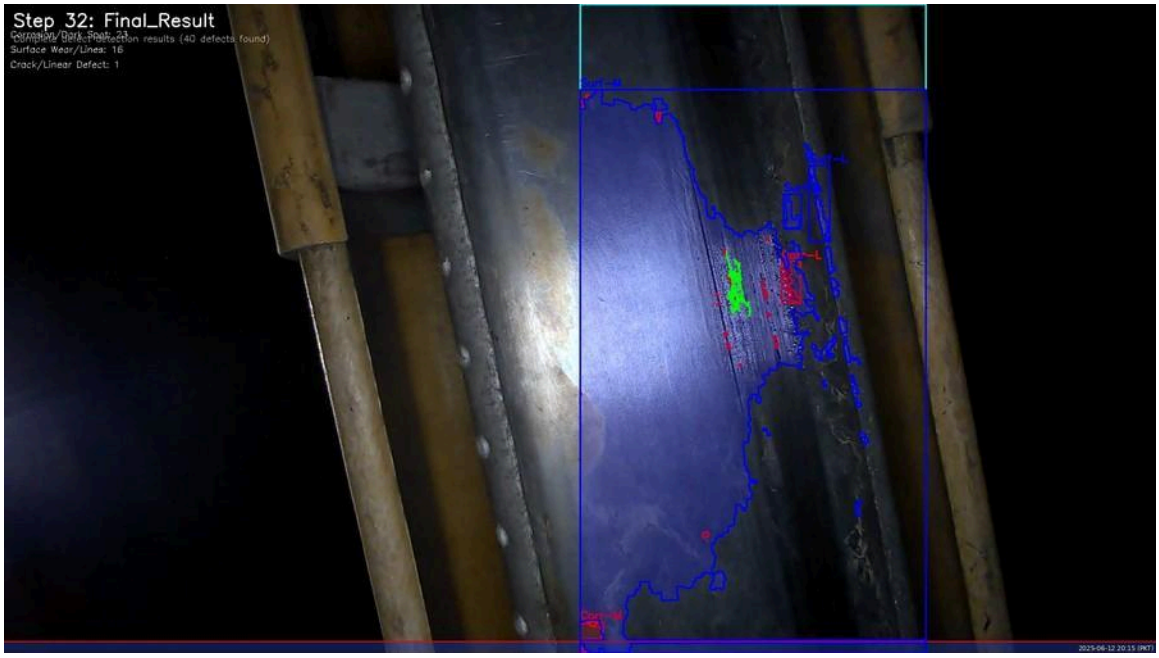

Figure 3 — Step 32: Final Detection Result — color-coded defect annotations with severity labels

3. Dataset & Annotation

3.1 Dataset Composition

A curated dataset of 54 real-world METRO Orange Line third-rail images was assembled and processed. The images capture diverse track sections, varying ambient lighting conditions, and multiple defect severities — ensuring the detection algorithms are validated against realistic operational conditions.

Dataset Property	Value	Notes
Total Images	**54**	Real METRO Orange Line imagery
Defect Types Covered	**3 categories**	Corrosion, Surface Wear, Cracks
Severity Levels	**3 levels**	Low, Medium, High
Image Resolution	**1920 x 1080 px**	HD quality input

3.2 Defect Classification Schema

Defect Type	Visual Indicator	Severity Trigger	Detection Method
Corrosion / Dark Spots	Red bounding box	Area > 800 px^2 (Med), > 2000 px^2 (High)	Adaptive + Simple Thresholding, Contour Analysis
Surface Wear / Lines	Blue bounding box	Area > 1500 px^2 (High)	High-Pass Filter, Morphological Line Detection, Laplacian
Cracks / Linear Defects	Green bounding box	Length > 100 px (High)	Canny Edge Detection, Probabilistic Hough Line Transform

4. Visual Results & Analysis Reports

The system generates three categories of output for every processed image: an annotated defect image, a side-by-side comparison, and a structured defect summary report. A selection of outputs from the dataset is presented below.

4.1 Side-by-Side Comparisons

Each comparison image places the original unannotated photograph alongside the fully processed result, allowing direct visual assessment of detection accuracy.

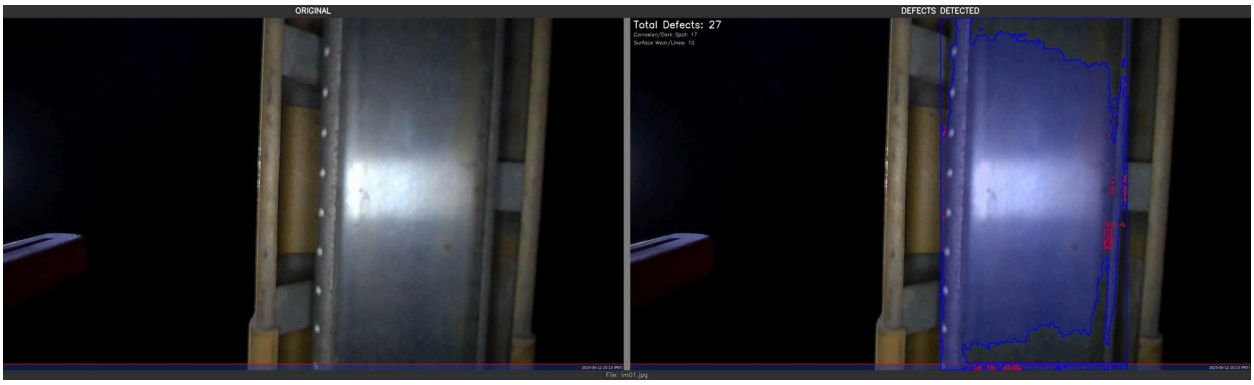

Figure 4 — Sample Comparison: im01 — Corrosion and wear patterns detected on Orange Line rail section

Figure 5 — Sample Comparison: im07 — Multiple defect categories detected simultaneously

Figure 6 — Sample Comparison: im45 — High-resolution detection on a heavily worn rail section

4.2 Sample Automated Defect Report

The system automatically generates a structured text report for every processed image. A representative excerpt from the demo run (40 defects detected on a single image) is shown below:

```
=== THIRD RAIL DEFECT DETECTION REPORT ===
Generated on: 2025-06-24 01:47:07
Total Defects Found: 40

Defect #1:  Type: Corrosion/Dark Spot  |  Severity: Low    |  Area: 51.00 px²
Defect #2:  Type: Corrosion/Dark Spot  |  Severity: Medium  |  Area: 904.00 px²
Defect #18: Type: Corrosion/Dark Spot  |  Severity: Low    |  Area: 659.50 px²
Defect #37: Type: Surface Wear/Lines   |  Severity: Medium  |  Area: 944.00 px²
Defect #39: Type: Surface Wear/Lines   |  Severity: Medium  |  Area: 230,328 px²
Defect #40: Type: Crack/Linear Defect  |  Severity: HIGH    |  Length: 109 px

[ 23 Corrosion  |  16 Surface Wear  |  1 Crack ]
```

5. System Performance

The TRDDS has been benchmarked in both batch and live processing modes. Performance metrics demonstrate viability for both laboratory demonstration and future embedded deployment on the Raspberry Pi 5 platform specified in the proposal.

Operating Mode	Resolution	FPS — Raspberry Pi 5	FPS — PC
Batch (TRD_training.py)	1920 x 1080 (original)	N/A	3–5 img/sec
Live (TRD_video.py)	640 x 480 (optimized)	**10–15 FPS**	**25–30 FPS**

The live mode achieves 10–15 FPS on Raspberry Pi 5 at 640x480 resolution — sufficient for real-time inspection as the camera traverses the track. Frame resolution can be increased once the physical scanning speed is calibrated in Phase 2.

6. Source Code & Repository

The complete source code has been published on GitHub and is packaged for future integration. All scripts are modular, fully commented, and include a comprehensive configuration section at the top of each file for easy tuning.

GitHub Repository: `https://github.com/meds-uet/third-rail-maintenance`

6.1 Repository Structure

Path / File	Contents
TRD_training.py	Batch image processing engine — processes folders, generates annotated outputs and comparisons
TRD_demo.py	32-step demo pipeline — saves every intermediate stage for presentation purposes
TRD_video.py	Live webcam / camera mode with GUI (RECORD, CAPTURE, EXIT buttons); Raspberry Pi 5 optimized
data/ (54 images)	Curated real METRO Orange Line third-rail images (im01.jpg – im55.jpg)
output/comparisons/	25 side-by-side before/after comparison JPGs
output/processed/	25 annotated defect detection output images
demo_output/	32 step-by-step pipeline stage images + defect_summary_report.txt
assets/	Sample original and processed images for documentation
Readme.md	Full installation, usage, configuration, and performance documentation

6.2 Installation & Dependencies

The system requires only two external Python libraries, both freely available:

- Python 3.8 or higher
- OpenCV 4.5+ — pip3 install opencv-python
- NumPy — pip3 install numpy

No proprietary libraries, cloud services, or subscription tools are required. The system is fully self-contained and runs offline — a critical requirement for trackside deployment.

7. Conclusion

Phase 1 of the Automated Third Rail Inspection System has been completed in full, on schedule, and within the agreed scope. All six deliverables defined in the original project proposal — the image processing pipeline, annotated dataset, visual reports, demo scripts, documentation, and packaged source code — have been produced and are presented in this report.

The system demonstrates reliable detection of the three primary defect categories present on the METRO Orange Line third rail: corrosion/dark spots, surface wear patterns, and cracks/linear defects. Detection results are automatically classified by severity, visualized with color-coded annotations, and compiled into structured reports.

The software is immediately ready for demonstration to METRO management and operations teams. Upon approval to proceed, Phase 2 hardware procurement can begin, paving the way for real-time embedded deployment as a permanent inspection asset for the Orange Line.

Contact & Payment Details

Umer Shahid
Lecturer, Department of Electrical Engineering
University of Engineering & Technology (UET), Lahore
umer.shahid@uet.edu.pk

Bank Transfer Details

Account Title: Umer Shahid
Bank: HBL — UET Lahore Branch
Account No.: 01287901788803
IBAN: PK02HABB01287901788803
SWIFT: HBLPKKA

Kindly notify via email once payment has been transferred. An official receipt will be issued accordingly.